Curiosity Report

A. Problem Exploration

1. The problem or opportunity your team identified

Students often need help understating material. The problem we identified is the lack of help they can get beside office hours or emailing the professor. We propose a connected discussion board for students in a class to ask questions they have regarding the subject at hand.

2. Who experiences the problem

Students are the main users who need assistance outside of class. But this also benefits professors as they might also get several emails about the same problem, now they can address the answer in one space instead of several separate emails.

3. When and where the problem occurs

When students are studying for an exam and doing homework outside of class.

4. How people currently address the problem & limitations of existing approaches

Some students create class group chats, which are inefficient since not all the class joins and questions can get buried under new messages. Students also email professors but it takes a while to get a response. Students also use professor office hours but often can't attend due to scheduling conflicts.

5. Why a networked application is appropriate

A network application is appropriate because multiple students and instructors need to access and update a discussion board from several different computers.

B. Curiosity Questions

1. Who would use this application most often?

This application will majority be used by students to get help for understanding core concepts and homeworks. Instructors would also use this application to answer questions, make sure the students are not spreading false information and get an understanding for what concepts students are struggling with.

2. What outcome is the user trying to achieve?

The user will be trying to get clear answers in a timely manner for course related questions. A user might post a question, find existing explanations or help answer other questions. The ultimate outcome for users is understanding class materials.

3. What information must be shared between users?

Users must be able to see titles of posts, content of posts, usernames of posters and see if a question was answered or is still open.

4. Which tasks must happen in real time?

The discussion board must be updated in real time with any new questions or answers that users may have submitted. By doing this the user does not have to refresh their side of the code in order to see new questions or answers.

5. What would cause a user to stop trusting the application?

Users could lose interest if posts somehow disappear or they keep losing connection to the server. In order to prevent this we made sure that even if a user loses connection suddenly they still have access to all the questions and answers once they log back in.

6. What is the minimum feature set that would still be valuable?

Simply being able to post a question and receive answers from peers would be the minimum feature set that would still be valuable. Without the professor being able to check the answers, or the discussion board moving questions that have been sufficiently answered to a different tab, even with just these features the system would still be very valuable to students.

7. What changes when the system grows from one user to 100 users?

A better sorting or search system may need to be implemented, as we could expect a higher intake of questions and answers. The system may also require more storage space, since there are more users, and thus more information to store.

8. Which data must persist after users disconnect?

The user's question and their answers must remain on the board even after the original poster has logged off. We did this by storing the information on the server side of the application. This way even if the client side disconnects all of the information remains. So as long as the server side stays running all of the information remains on the dashboard.

C. User & Stake Holder Evidence

One existing product we found to be similar to ours is WhatsApp. During a short interview with another student we found that it is common for classes to create group chats in WhatsApp for each of their classes. We asked the student if they thought these group chats were an effective way to ask questions about the class. They did not feel confident in these group chats' ability to be a good way to get important questions answered. They said that since often people just use the group chats as a social outlet, questions about the class often go unanswered. They proceeded to express how difficult it is to find a previously asked question. Since people talk in these chats so much, there are a lot of chats to sift through to find what you are looking for. We solve this problem, since our dashboard is solely used for asking questions there is less

chance of questions being buried by walls of text. Furthermore, questions and answers are the sole focus of our application, so there is no socializing to distract from this goal.

D.Targest Users
Primary users: Students High School and Up
Secondary users: Professors and Teachers
User goals: Users want to……

- Ask question about course material to receive help in a timely manner
- Find answers to previously answered questions
- Share helpful information with peers

Likely user frustrations: Users can get frustrated…….

- When their questions are not getting answered
- When false or misleading information is spread
- If application is too complicated to figure out

Technical experience assumptions: We assume users have basic computer skills and it should be relatively easy for any user who is able to work a website such as canvas.

Accessibility considerations: We should make the system very easy to navigate with readable formatting and clear menu options.

User Stories:

1. As a student, I want to be able to have somewhere I can ask a specific question about a class and receive an answer, so that I am not stuck getting unrelated answers off of trying to google my question.
2. As a student, I want to be able to review existing questions so that I can find help without sending out the same question.
3. As a student, I want to be able to tell my classmates an easy way I have found to solve a type of problem, so that everyone is able to see it.

E.Value Proposition

For students, who need a question answered using the class's teachings, our application provides a discussion board where students can help other students. Unlike a WhatsApp chat, our discussion board will be specifically for questions and answers.

Functional Value: Allows students to post course related questions, view existing discussions and respond to classmates. It allows for professors to cut down on answering the same question multiple times.

Experimental Value: Allow students to receive answers based on methods and techniques used and taught by the class rather than general web answers which may be different or have different approaches.
Reliability Value: Users will be able to rely on the answers they receive from our system, since they are from people taking the same class as them and thus will be solving the problems the same way the professor is teaching them to.

F.Assumptions, Risk & Open Questions

Assumptions

- We assume Students will ask questions on Board
- We assume Students will answer questions posted on Board
- We assume Professor will check discussion posts

Risks and Mitigation

- There is a risk that some users will only read posts and not answer people's questions. Low participation could lead to less student usage. Some mitigation could be to make it easy to post and answer. Encouragement to use application by the professor could work to minimize this too.
- A client could disconnect unexpectedly. To mitigate this the server will make sure to save at the last state the client was at. The client will also display a clear message if the server disconnects.
- A risk is that students may post misleading or false information. Our solution is to encourage users to check the information they find through our application with work given by the professor to ensure that it is correct.

Unanswered Questions

- How will this system be tied to the university class? Whatsapp groups are usually made by students or TAs since this one includes the professor, will it be tied through canvas or a similar site?
- What will happen to the discussion board once the class is over? Will it be archived, or would it be completely deleted?

Problem Statement

Students who do not understand a concept in a class they are taking do not have a great venue to request help from as of now. Asking for help in a class group chat can result in their question getting lost in an avalanche of other messages, whereas searching for an answer on the web can result in answers unrelated to the student's course. Students need a space with their peers where they can ask questions and receive answers relevant and correct.

Value Proposition

DiscussU is a more reliable way to answer a user's questions than, say, a class group chat because here the focus is entirely on questions and answers, so no question will get lost in the noise. As DiscussU will have separate discussion boards for different classes, a user will be able to get relevant answers to their questions within the scope of the class they are in.

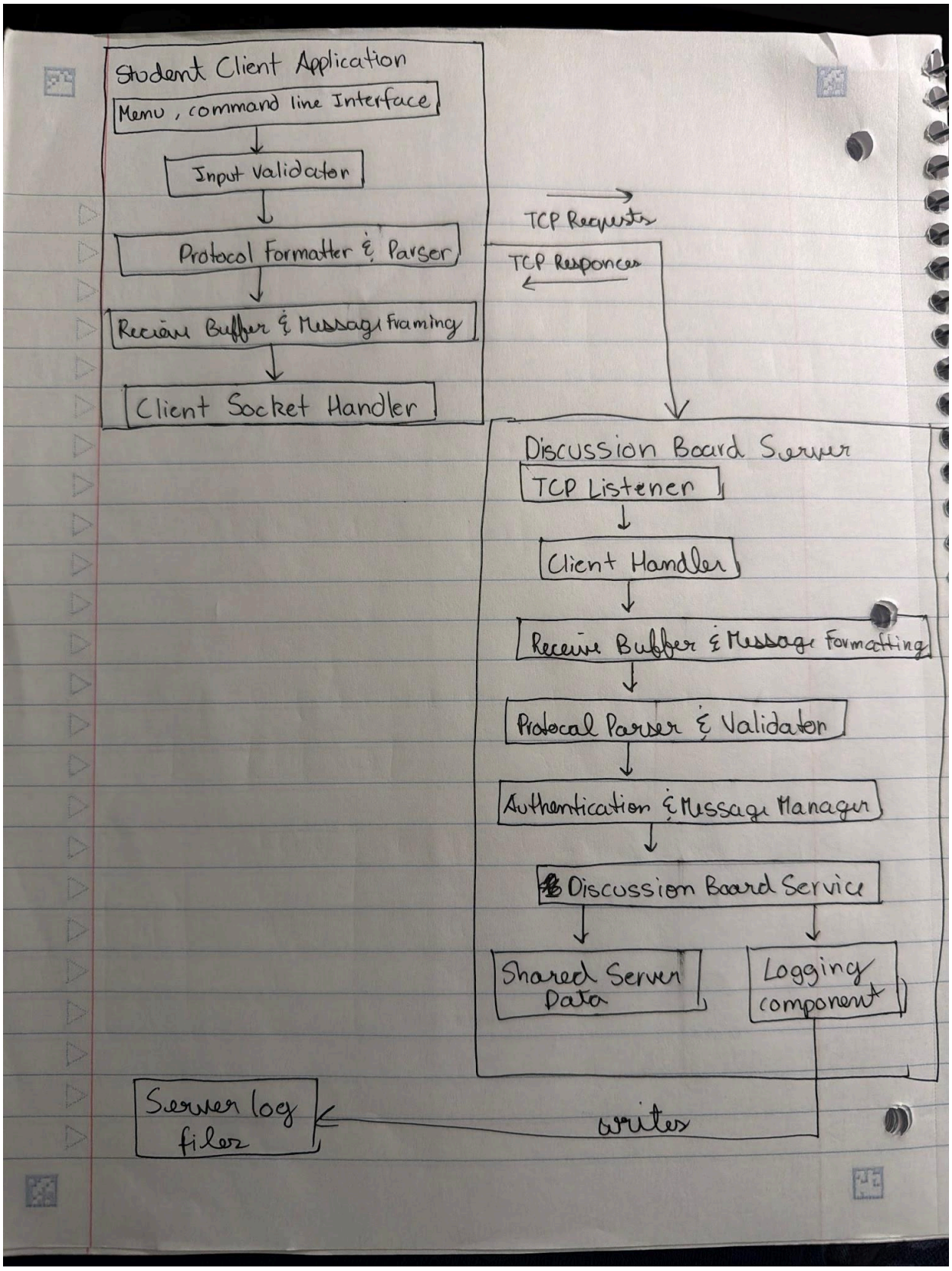
Final Requirements

FR-01: Client will allow the user to view the discussion board
FR-02: Client will allow the user to post a question to the discussion board
FR-03: Client will allow the user to reply to another user's question with an answer
FR-04: Client must link multiple users into one discussion board they can all view
FR-05: Client will allow multiple users to update the discussion board at same time.
FR-06: Client will update the discussion board with new questions and answers posted by users, and make it visible to all.
FR-07: System will check to ensure that a user's post has been sent to the server and logged.
FR-08: System will reject unsupported client requests, synch as incomplete or ill-formatted request, without terminating server or deleting user information.

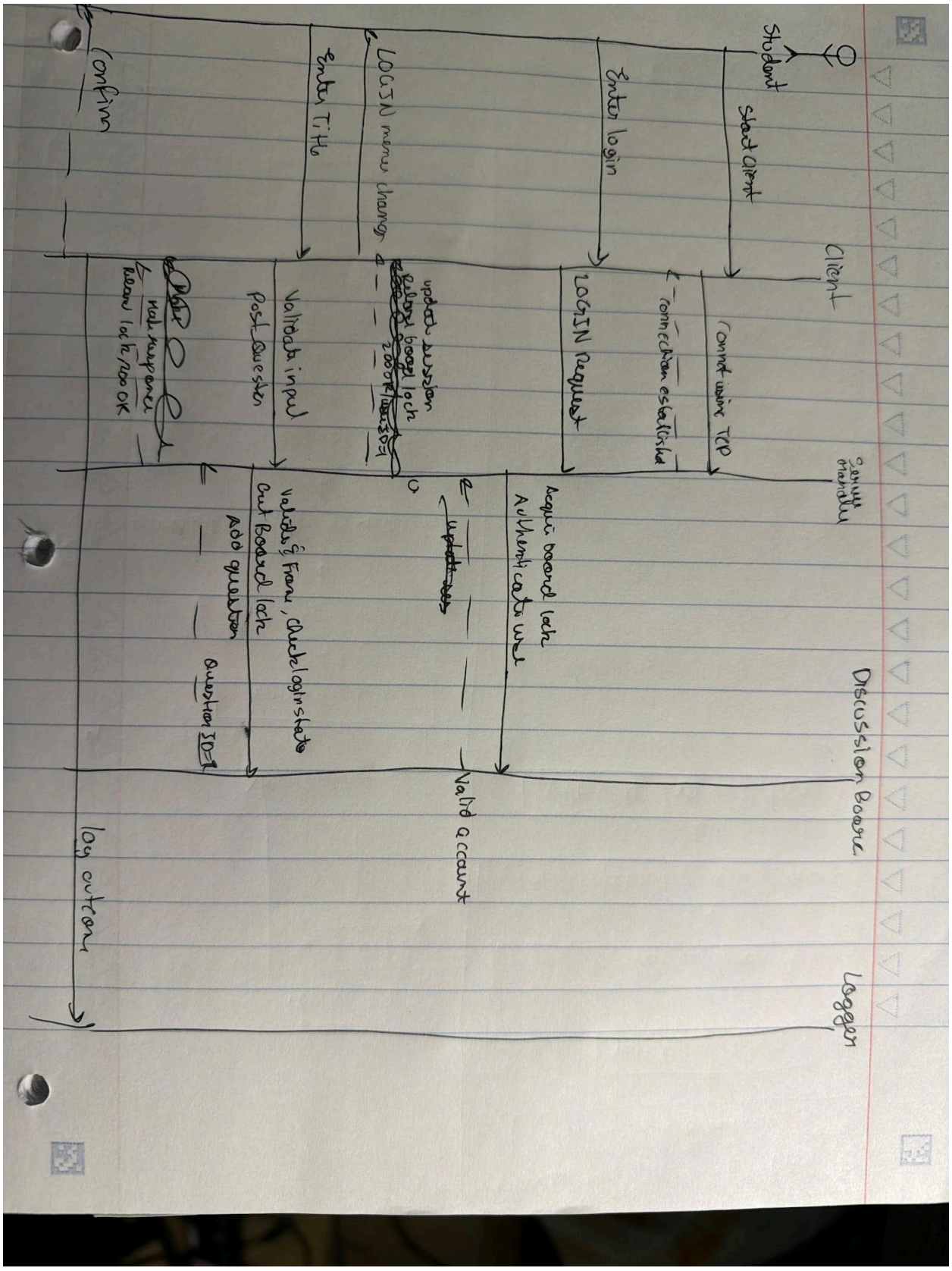
Nonfunctional Requirements

NFR-01: A user will be able to identify how to post a question and answer with ease.
NFR-02: The system will update the discussion board with new posts in a second or less.
NFR-03: The system will be able to handle multiple users making posts.
NFR-04: The system will have code simple enough to edit so it may be updated with relative ease when need be.
NFR-05: The system will only allow students in the class to be a part of a discussion board.
NFR-06: The system will use a user friendly font and layout to ensure all users are able to read it with ease.

Architecture Diagram



Sequence Diagram



Concurrency Design

DiscussU uses one handler thread per connected client. The main server thread accepts connections and starts a separate handleClient() thread for each client. This approach was chosen because it builds on the Phase 2 design as it allows multiple students to use the board without waiting for another client to disconnect.

Each handler maintains its own login session and receive buffer. All handlers share the same DiscussionBoard, including questions, answers, and ID counters. A shared board mutex protects the complete request-processing operation, preventing lost updates and duplicate IDs. The lock is released before sending the response, so a slow network connection does not hold the board lock.

The ServerLogger uses a separate mutex to prevent concurrent log entries from becoming mixed. Board and logging locks are not held together, reducing the risk of deadlock.
When two clients answer the same question, their updates are processed one at a time, and both valid answers are retained with different IDs. When a client leaves, its handler closes the socket and releases its session and buffer. During server shutdown, handlers check the shutdown flag, close their connections, and finish. The server joins all handler threads before destroying shared data and logging resources.

Protocol Specification

Overview

The discussion board is a line-oriented client-server protocol. All of the commands and responses are transmitted as ASCII text and they are terminated by a newline character such as \n.

Our system is a discussion board for students to be able to make queries about their class, and have them answered by other students in the class. Students will be able to login, post a question or answer, and view other students' questions or answers.

The server must allow the client to issue any of the following commands:

- LOGIN
- LOGOUT
- VIEW_QUESTIONS
- VIEW_ANSWER
- POST_QUESTION
- POST_ANSWER

Response Codes

Code	Meaning
200 OK	The command was accepted and run without error.
300 Missing/Incorrect Required Fields	The command entered had invalid info for a required field
401 Login is required	The entered command requires the user to be logged in first.
404 Not Found	The requested destination does not exist.
410 Wrong username or password	Credentials entered are not valid.
413 Content too Large	The message exceeds the maximum message size of 4096 bytes.

Message Framing

Each message will be sent as a line of text ending with a newline character. The maximum message size is 4096 bytes.

Character Encoding

All messages between the client and the server will use ASCII encoding.

Connection Initialization

When a student would like to connect to the server they must

1. Establish a connection to the server.
2. They will then use the LOGIN command.
3. The server authenticates the clients entered username and password.
4. The server will either respond with a 200 OK or 410 Wrong username or password.
5. If the login is successful, the client will enter the Authenticated session state.

Session States

1. Connected: In this state the client has connected to the server but has not logged in.
   Allowed Commands
   - LOGIN
2. Authenticated: In this state the client has logged in.
   Allowed Commands
   - VIEW_QUESTIONS
   - VIEW_ANSWER
   - POST_QUESTION
   - POST_ANSWER
   - LOGOUT
3. Disconnected: In this state the client has logged out of the server.

If the client tries to enter a command that is not permitted in the current state the server returns an error.

Normal Disconnection

A client may disconnect from the server by entering the LOGOUT command while in the Authenticated state.

Invalid Message Handling

The server will authenticate each command before processing it. The server will return an Error message if

- The command is unknown
- The command is missing a required field
- A field contains an incorrect data type
- A field contains an incorrect data value
- The message is larger than 4096 bytes
- The message contains non ASCII characters
- The message is sent during an invalid session state

Commands

LOGIN
Sender: Client
Receiver: Server
Required Fields:

- Username = string
- Password = string

Valid Values:

- Username: A student's valid username
- Password: must only contain ASCII characters

Example Request: LOGIN |username= KieranF| password= Pro123\n
Example Success Response: 200 OK | userID= 1\n

If the client entered an invalid username or password
Example Error Response: 410 Wrong username or password\n

LOGOUT
Sender: Client
Receiver: Server
Example Request: LOGOUT\n
Example Success Response: 200 OK\n

If the client tried to logout before they were logged in
Example Error Response: 401 Login is required\n

VIEW_QUESTIONS
Sender: Client
Receiver: Server
Example Request: VIEW_QUESTIONS\n
Example Success Response:

- 200 OK\n
- A List of all posted questions\n

Each question will be posted on a separate line

If a client is not logged in when they run this command
Example Error Response: 401 Login is required\n

VIEW_ANSWER
Sender: Client
Receiver: Server
Required Fields:
- Question_Number= integer

Valid Value
- Question_Number: 2

Example Request: VIEW_ANSWER | 2\n
Example Success Response:
- 200 OK\n
- Question_Number= 2 | Author= Bob | Title= What is a unicorn? | Body= Guys what is a unicorn? | answers= 1\n

If a client attempted to view the answer of a question that did not exist
Example Error Response: 404 Not Found\n

POST_QUESTION
Sender: Client
Receiver: Server
Required Fields:
- Title= string
- Body= string

Valid Values:
- Title= What is a unicorn?
- Body= Guys what is a unicorn?

Example Request: POST_QUESTION | Title= What is a unicorn? | Body= Guys what is a unicorn?\n
Example Success Response: 200 OK\n

If a client entered no data for the title or body
Example Error Response: 300 Missing/Incorrect Required Fields\n

POST_ANSWER
Sender: Client
Receiver: Server
Required Fields:
- Question_Number= integer
- Answer= string

Valid Values:
- Question_Num: 2

- Answer: A unicorn is a mythical creature that resembles a horse with a horn on its head.

Example Request:POST_ANSWER | Question_Number= 2 | Answer= A unicorn is a mythical creature that resembles a horse with a horn on its head.\n
Example Success Response: 200 OK\n

If a client tried to enter a question number that did not exist
Example Error Request: 404 Not Found\n

Versioning

When the code is majorly updated we can increase the version by 1. The system will start as Version 1 and will increase as the code is updated more and more times.

Server Notifications

The server will not be able to send unsolicited notifications. For instance when a user posts a question, they must run the correct command sequence to view the responses to their question, the server will not show it automatically.

Data Design

Overview: The client will store the information needed for the current connection and logged in session for example session state and server address. While server stores, user accounts, active session, questions, answers and unique IDs for various data.

Classes:

1.ClientState

- Purpose: Stores information needed by one client applications when it is connected to the server
- Fields and data types
    - serverHost: String → server hostname or Internet Protocol address
    - serverPort: Integer → server TCP port
    - userId: Integer → authenticated user ID returned by the server
    - username: String → authenticated user's username
    - sessionState: SessionState → current connection and authentication state
- Ownership: Client
- Validation Rules:
    - serverHost must not be empty
    - serverPort must be a a valid integer and fit in post serve listed
    - userId must be a valid integer
    - username must contain only permitted ASCII characters
- Mutable: Yes because status changes when the client connects, logs in, logs out, or loses its connection.

2.AuthenticatedSession

- Purpose: Stores data required to authenticate a user/student
- Values
    - sessionId: String → unique server-generated session identifier
    - userId: Integer → ID of the authenticated user
    - username: String → authenticated username
    - sessionState: SessionState →current state of the session
    - connected: Boolean → whether the TCP connection is active
    -
- Ownership: Server
- Validation Rules:
    - userID must be unique positive integer
    - username must be nonempty and unique for each student
    - username and password must be made with permitted ASCII characters
- Mutable: No

3. UserAccount

- Purpose: Associates an authenticated user with a specific active TCP client connection.
- Fields and data types:
    - userID: Integer → unique student identifier
    - username: String → login name
    - Password: String → authenticated password
- Ownership: Server
- Validation Rules:
    - sessionId must be unique and nonempty.
    - userId must refer to an existing UserAccount.
    - A session cannot become AUTHENTICATED until the server validates the username and password.
    - Authentication applies only to the client connection that submitted the credentials.
    - The session must be removed when the client logs out or disconnects.
    -
- Mutable: Yes. The session state and connection status can change.

4.Answer

- Purpose: Represents an answer posted in response to a question.
- Fields and data types:
    - answerId: Integer → unique answer identifier
    - questionNumber: Integer → number of the associated question
    - authorUserId: Integer → ID of the student who posted the answer
    - authorUsername: String → username of the answer's author
    - body: String → answer text
- Ownership: Server
- Validation Rules:
    - answerId must be a unique positive integer.
    - questionNumber must refer to an existing question.
    - The author must have an authenticated session.
    - The body cannot be empty.
    - body must contain only ASCII characters.
- Mutable: No

5.Question

- Purpose: Represents a question posted on discussion board
- Fields and data types:
    - questionNumber: Integer → unique question number assigned by the server
    - authorUserId: Integer → ID of the student who posted the question
    - authorUsername: String →username of the author
    - title: String → short question title
    - body: String → complete question text
- Ownership: Server
- Validation Rules:
    - questionNumber must be a unique positive integer.
    - The author must have an authenticated session when posting.
    - title and body cannot be empty.
    - title and body must contain only ASCII characters.
- Mutable: Yes because answers may be added to questions.

6.DiscussionBoard

- Purpose: Stores and manages all shared server data.
- Fields and data types:
    - userAccounts: Map<String, UserAccount> → maps usernames to account records
    - activeSessions: Map<String, AuthenticatedSession> → maps session IDs to current sessions
    - questions: Map<Integer, Question> → maps question numbers to questions
    - nextQuestionNumber: Integer → number assigned to the next question
    - nextAnswerId: Integer → ID assigned to the next answer
    - Main operations:
        - authenticate(username, password)
        - logout(sessionId)
        - viewQuestions(sessionId)
        - viewAnswer(sessionId, questionNumber)
        - postQuestion(sessionId, title, body)
        - postAnswer(sessionId, questionNumber, body)
- Ownership: Server
- Validation Rules:
    - Only authenticated sessions may access discussion-board commands.
    - Usernames, question numbers, and answer IDs must be unique.
    - Referenced questions must exist.
    - Required fields must be present and use valid values.
    - Invalid requests must not partially modify shared data.
    - Credentials must not be included in responses or logs.

7.ProtocolMessage:

- Purpose: Represents one command or response exchanged between the client and server.
- Fields and data types
    - command: String → command name, such as LOGIN or POST_QUESTION
    - fields: Map<String, String> → fields included with the command
    - responseCode: Integer → response code, such as 200, 300, 401, 404, or 410
    - responseText: String → explanation associated with the response
    - rawMessage: String → complete message before parsing
- Ownership: Shared between the client and server
- Validation Rules:
    - Every message must end with a newline character (\n).
    - Every message must use ASCII encoding.
    - A message cannot exceed 4,096 bytes.
    - The command must be recognized by the server.
    - Required fields must be present.
    - Fields must use the expected data types and values.
    - The command must be allowed in the current session state.
- Mutable: No

8.ServerLogEntry

- Purpose: Represents one event recorded in the server log.
- Fields and data types:
    - timestamp: DateTime → time when the event occurred
    - level: String → INFO, WARN, or ERROR
    - sessionId: String → associated session ID, when applicable
    - command: String → command being processed, when applicable
    - outcome: String → result of the event
    - resourceId: Integer → related question or answer ID, when applicable
- Ownership: Server
- Validation rules:
    - Every entry must contain a timestamp, level, and outcome.
    - Passwords must never be written to the log.
    - Full question and answer text should not be logged unnecessarily.
    - Question and answer IDs may be logged for troubleshooting.

Mutable: No

Structures

1.ServerState:

- Purpose: Represents current state of the clients server connection.
- Vales of state
    - CONNECTED → connected to server but no authenticated
    - AUTHENTICATED → successfully logged in
    - DISCONTINUED → no active server connection
- Ownership: Client and Server
- Validation Rules:
    - LOGIN is an action permitted within CONNECTION
    - LOGIN, LOGOUT, VIEW_QUESTIONS, VIEW_ANSWER, POST_QUESTION, POST_ANSWER are permitted in authentication state
    - After LOGOUT, the sate of serves connection becomes DISCONTED
- Mutable: Yes because the state assigned to the client serves changes.

Persistence

The minimum viable product does not use permanent storage for questions and answers.

- User accounts are loaded from a server-controlled source.
- Active sessions exist only while clients are connected.
- Questions and answers remain in server memory while the server is running.
- Questions and answers are lost when the server shuts down or restarts.
- Server activity is written to a log file.
- The log file is not used to restore questions or answers.

Permanent storage for questions and answers is future work.

Error Handling Strategy

DiscussU checks the inputs of client and server. The client rejects invalid menu choices, empty fields, unsupported characters, invalid question numbers, and oversized requests before sending them. It displays clear messages and lets users correct their input. If terminal input ends, the client exits instead of looping indefinitely.

And the server independently validates every request. This includes command names, required fields, field types, session state, ASCII encoding, and the 4,096-byte line limit. Invalid requests receive structured error responses, such as 300 for malformed messages, 401 when login is required, and 404 for missing questions. Rejected requests do not change discussion-board data.

Our error handling strategy is, if an incorrect command is entered then the user will receive an error message telling them what was wrong and they will be prompted for another command. If the user enters incorrect information after a correct command the same thing occurs. They will again be prompted to re enter correct information. In both of these cases, we made it so the information is not sent to the server so that errors are not logged in the server log.

Test Results Phase 3

Identification

Project: DiscussU
Phase: 3 — Multiple Clients and One Server
Team Members: Jahan Clare and Kieran Fix
Tested By: Jahan Clare
Test Script: tests/protocol_tests.py

2. Test Environment

Operating System: Linux in GitHub Codespaces
Programming Language: C++17
Automated Test Language: Python 3
Network: Client and server communication over localhost
Server Port: 6543
Concurrency Approach: One handler thread per connected client

The automated script creates separate TCP client connections. It uses persistent receive buffers to read newline-delimited responses and continues reading successful view responses until END.

3. Scope and Approach

We tested DiscussU using an automated Python script against the running C++ server. The script checked session-state rules, malformed requests, error recovery, message framing, shared application data, concurrent answers, unexpected disconnections, repeated connections, and concurrent load performance.

A small run used three load-test clients with five posting requests each. The main captured run used ten load-test clients with twenty posting requests each.

The automated script tests the server directly through Python TCP clients. It does not test the C++ client's menu presentation or terminal-input handling. Those behaviors, along with server shutdown and restart, are documented separately through manual tests.

4. Summary of Results

Automated Test Group	Status
Session-state validation	Pass
Invalid requests and recovery	Pass
Fragmented request	Pass
Batched requests	Pass
Three connected clients performing different operations	Pass

Concurrent answers retained with unique IDs	Pass
Invalid request and disconnect isolation	Pass
Repeated connections and graceful QUIT	Pass
Concurrent load test	Pass
Total	**9 passed, 0 failed**

5. Detailed Automated Test Cases

Screenshot:



```
TEST: Fragmented request
  Sent fragment 1: VIEW_QUES (without newline)
  No response before the newline
  Sent fragment 2: TIONS\n
  Received one complete response ending with END
PASS: Fragmented request

TEST: Batched requests
  Sent two VIEW_QUESTIONS requests together
  Received two complete responses ending with END
PASS: Batched requests

TEST: Three connected clients perform different operations
  Three clients are connected and authenticated
  Client 1 created question 23
  Client 2 retrieved Client 1's question
  Client 3 posted answer 6
  Client 1 retrieved Client 3's answer
PASS: Three connected clients perform different operations

TEST: Concurrent answers retained with unique IDs
  Stored 3 answers with unique IDs: [8, 9, 7]
PASS: Concurrent answers retained with unique IDs

TEST: Invalid request and disconnect isolation
```

```
TEST: Invalid request and disconnect isolation
  Invalid request rejected; all connections remain usable
  One client closed without QUIT
  Remaining clients posted and retrieved both questions
PASS: Invalid request and disconnect isolation

TEST: Repeated connections and graceful QUIT
  Completed 15 connect/login/view/QUIT cycles
PASS: Repeated connections and graceful QUIT

TEST: Concurrent load test

LOAD TEST RESULTS
Clients: 10
Planned posting requests: 200
Successful posting requests: 200
Failed/not completed posting requests: 0
Total duration including connection/cleanup: 0.098 seconds
Average completed-request response time: 2.355 ms
Maximum completed-request response time: 8.305 ms
Verified that all 200 created questions were retained.
Verification requests are additional to the timed posting workload.
PASS: Concurrent load test
```

```
FINAL TEST SUMMARY
Total test groups: 9
Passed: 9
Failed: 0
```

P3-A01 — Session-State Validation

Requirement Tested: The server enforces login-state rules separately for each connection.

Initial Conditions: A test client is connected but not authenticated.

Input or Action:

1. Request VIEW_QUESTIONS before login.
2. Log in using a valid configured account.
3. Attempt LOGIN again while authenticated.
4. Perform a valid view request.
5. Log out and request VIEW_QUESTIONS again.
6. Log in again and send QUIT.

Expected Result:

- VIEW_QUESTIONS before login returns response code 401.

- Valid login succeeds.
- A second LOGIN returns response code 409.
- The authenticated session remains usable after rejection.
- After logout, board access again returns 401.
- The same connection can log in again and quit gracefully.

Actual Result: The server returned 401 before login and after logout. Valid login succeeded, and a second login returned 409. Subsequent valid operations and graceful QUIT succeeded.

Status: Pass

Evidence: Session-state validation output in tests/phase3-results.txt.

P3-A02 — Invalid Requests and Recovery

Requirement Tested: The server rejects invalid requests without corrupting data or making the connection unusable.

Initial Conditions: The server is running. The script authenticates a client and creates a question for recovery checks.

Input or Action: Submit:

- An incorrect password
- An empty request
- An unknown command
- A question request missing its body
- A question request with an empty title
- A duplicate field
- An unexpected field
- A nonnumeric question number
- A zero question number
- A request for a nonexistent question

Expected Result:

- Incorrect credentials return 410.
- Empty, missing, duplicate, unexpected, and invalid fields return 300.
- An unknown command returns 400.
- A nonexistent question returns 404.
- Valid requests continue working after the rejected requests.

Actual Result: All expected response codes were returned. The script verified successful recovery requests after the malformed requests and confirmed continued operation after the nonexistent-question response.

Status: Pass

Evidence: Invalid requests and recovery output in tests/phase3-results.txt.

P3-A03 — Fragmented Request

Requirement Tested: The server does not process an incomplete message before its terminating newline arrives.

Initial Conditions: A test client is connected and authenticated.

Input or Action:

1. Send VIEW_QUES without a newline.
2. Check for a response during a 0.3-second observation interval.
3. Send TIONS followed by a newline.
4. Read the complete response through END.
5. Send a subsequent valid request.

Expected Result:

- No response arrives before the terminating newline.
- The fragments form one VIEW_QUESTIONS request.
- One complete response is returned.
- The connection remains usable afterward.

Actual Result: No response arrived before the newline. After the second fragment was sent, the server returned one complete response ending with END. The following request and QUIT also succeeded.

Status: Pass

P3-A04 — Batched Requests

Requirement Tested: Multiple newline-delimited requests sent together are processed separately and in order.

Initial Conditions: A test client is authenticated.

Input or Action: Send the following bytes together:

VIEW_QUESTIONS\nVIEW_QUESTIONS\n

Read two complete responses, then send QUIT.

Expected Result:

- Two complete responses are returned.
- Each begins with 200 OK and ends with END.
- The unchanged-board responses match.
- QUIT succeeds afterward.

Actual Result: The script received two matching, complete responses ending with END. Graceful QUIT succeeded.

Status: Pass

P3-A05 — Three Connected Clients Performing Different Operations

Requirement Tested: Multiple clients can access and update the same shared board.

Initial Conditions: Three separate clients are connected and authenticated.

Input or Action:

1. Client 1 posts a question.
2. Client 2 retrieves that question.
3. Client 3 posts an answer.
4. Client 1 retrieves the question and its answer.
5. All clients quit.

Expected Result:

- All three connections remain active during the operations.
- Each client can access data created by another.
- The answer count is one, and the saved answer text is correct.

Actual Result: Client 1 created Question 23. Client 2 retrieved it. Client 3 posted Answer 6, and Client 1 retrieved the saved answer. All assertions passed.

Status: Pass

P3-A06 — Concurrent Answers to the Same Question

Requirement Tested: Overlapping updates preserve every valid answer and assign unique IDs.

Initial Conditions: A newly created question has no answers.

Input or Action:

1. Connect and authenticate three worker clients.
2. Use a barrier to make all workers ready before posting.
3. Submit three different answers to the same question.
4. Retrieve the question and verify the saved answers.

Expected Result:

- All three answers succeed.
- Answer IDs are unique.
- Every submitted answer is retained.
- The answer count is three.

Actual Result: The server stored all three answers with IDs 8, 9, and 7. The script verified all answer bodies and an answer count of three.

Status: Pass

Interpretation: The IDs need not follow worker-number order because threads may acquire the board lock in different orders.

P3-A07 — Invalid Request and Disconnect Isolation

Requirement Tested: One client's error or unexpected disconnection does not interrupt other clients.

Initial Conditions: Three clients are connected and authenticated.

Input or Action:

1. One client submits a POST_QUESTION request missing its body.
2. Verify that all three clients can still perform valid requests.
3. Close the affected client without sending QUIT.
4. Have the remaining clients post and retrieve questions.

Expected Result:

- The malformed request returns 300.
- All connections remain usable after the error.
- The remaining clients continue working after the unexpected disconnect.
- Their questions are retained with different IDs.

Actual Result: The malformed request was rejected, and all three connections remained usable. After one client closed without QUIT, both surviving clients successfully posted and retrieved questions.

Status: Pass

P3-A08 — Repeated Connections and Graceful QUIT

Requirement Tested: The server remains available across repeated connection cycles and closes sockets after QUIT.

Initial Conditions: The server is running.

Input or Action: Perform fifteen cycles of:

1. Connect
2. Log in
3. View questions
4. Send QUIT
5. Verify connection closure

Expected Result: Every cycle completes, each view response ends with END, and QUIT is confirmed before the server closes the connection.

Actual Result: All fifteen connection/login/view/QUIT cycles completed successfully.

Status: Pass

Limitation: This test verifies functional cleanup behavior. It does not directly measure memory usage or prove that all resource leaks are absent.

P3-A09 — Concurrent Load Test

Requirement Tested: Concurrent request processing, unique ID generation, data retention, and measured performance under a modest workload.

Initial Conditions: The server is running. Ten worker clients use a configured test account.

Input or Action:

1. Connect and authenticate ten clients.
2. Synchronize workers before posting.
3. Have each client submit twenty questions.
4. Measure completed-request response times.
5. Verify unique question IDs.
6. Retrieve every successfully created question afterward.
7. Terminate the test connections.

Expected Result:

- All 200 timed posting requests succeed.
- Question IDs are unique.
- Every created question is retained and retrievable.
- The server remains operational.

Actual Result: All 200 timed posting requests succeeded. No posting requests failed or remained incomplete. The script verified unique IDs and retrieved all 200 created questions.

Status: Pass

6. Load-Test Measurements

Command: python3 -u tests/protocol_tests.py localhost 6543 --clients 10 --requests 20 2>&1 | tee tests/phase3-results.txt

Measurement	Result
Concurrent load-test clients	10
Planned timed posting requests	200
Successful posting requests	200
Failed or incomplete posting requests	0
Total duration including connection and cleanup	0.098 seconds
Average completed-request response time	2.355 milliseconds

8. Manual Tests

P3-M01 — Invalid menu Input

Requirement Tested: The server reports a missing question without closing the connection.

Initial Conditions: The client is authenticated, and Question 1 does not exist.

Action: Select View Question and Answers and enter question number 1.

Expected Result: The server returns 404 question not found. The client remains connected and allows another operation**.**

Actual Result: The client displayed 404 question not found and returned to the discussion-board menu.

```
1. View questions
2. View question and answers
3. Post question
4. Post answer
5. Logout
6. Quit
Selection: 2
Question number: 1
404 question not found

Discussion Board Menu:
```

Status: Pass

P3-M02 — Ctrl+D handling

Requirement Tested: The client handles end-of-input without looping indefinitely.

Initial Conditions: The client is connected and displaying the discussion-board menu**.**

Action: Press Ctrl+D at the empty Selection prompt.

Expected Result: The client reports that input has ended, closes its socket, and exits without repeatedly displaying the menu.

Actual Result: The client displayed “Input ended. Closing client.”

```
Discussion Board Menu:
1. View questions
2. View question and answers
3. Post question
4. Post answer
5. Logout
6. Quit
Selection:
Input ended. Closing client.
```

Status: Pass

P3-M03 — Invalid Port

Requirement Tested: The client rejects an out-of-range port without attempting a connection.

Initial Conditions: The client is not running.

Action: Run: ./client localhost 666666

Expected Result: The client displays “Invalid port.” and exits.

Actual Result: The client displayed “Invalid port.” and returned to the shell prompt.

```
@jahanCl ➜ /workspaces/CIS427_SocketProgrammingProject_Group2 (fix-logic) $ ./client localhost 6666666
Invalid port.
```

Status: Pass

P3-M04 — Invalid Menu Input

Requirement Tested: Invalid menu selections are rejected locally without being sent to the server.

Initial Conditions: The client is connected and displaying a menu.

Action: Enter an invalid 999

Expected Result: The client displays an invalid-selection message, shows the menu again, and successfully processes the later valid choice.

Actual Result:

```
Discussion Board Menu:
1. View questions
2. View question and answers
3. Post question
4. Post answer
5. Logout
6. Quit
Selection: 999
Invalid menu selection. Please try again.

Discussion Board Menu:
1. View questions
2. View question and answers
3. Post question
4. Post answer
5. Logout
6. Quit
Selection:
```

Status: Pass

P3-M05 — Server Unavailable

Requirement Tested: The client reports connection failure without crashing.

Initial Conditions: No server is listening on port 6543.

Action: Run:./client localhost 6543

Expected Result: The client reports that it cannot connect and exits without crashing**.**

Actual Result:

```
@jahanCl ➜ /workspaces/CIS427_SocketProgrammingProject_Group2 (fix-logic) $ ./client localhost 6543
Unable to connect to server.
```

Status: Pass

P3-M06 — Oversized input recovery

Requirement Tested: Oversized user input is rejected locally, and the client remains usable.

Initial Conditions: The client is authenticated.

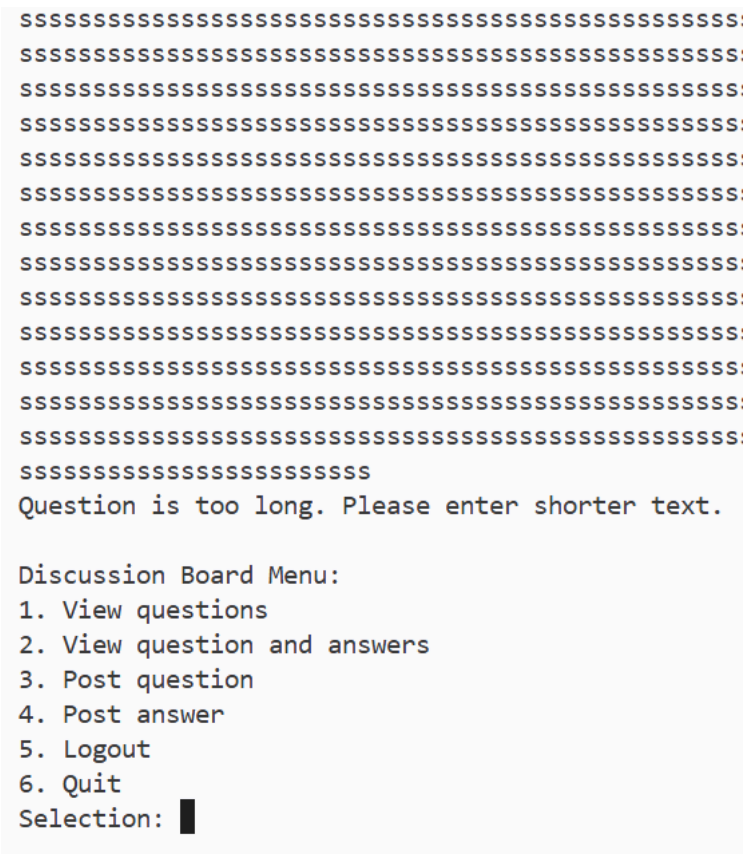
Action:

1. Select Post Question.
2. Enter a title.

3. Enter a body that makes the complete request exceed 4,096 bytes.

Expected Result: The client rejects the oversized request, returns to the menu, and successfully submits the shorter question. No oversized question is stored.

Actual Result: The screenshot shows "Question is too long. Please enter shorter text."



Status: A little. It doesn't close the client, but it doesn't return to questions. It returns to menu

P3-M07 — Server Restart Behavior

Requirement Tested: The server can restart on the same port and follow its documented in-memory storage behavior.

Initial Conditions: At least one question exists before a graceful shutdown**.**

Action:

1. Shut down the server.
2. Restart it using ./server 6543.
3. Connect and log in.
4. View questions.

   Expected Result: The server listens again, login succeeds, and the board is empty because questions and answers are not persisted**.**

Actual Result: Server disconnected. Before we added question Blue. Reconnecting client and no such question found.

```
Selection: 3
Title: Blue
Question: Why is sky blue
Question posted successfully.

Discussion Board Menu:
1. View questions
2. View question and answers
3. Post question
4. Post answer
5. Logout
6. Quit
Selection: 1
The server disconnected.
@jahanCl ➜/workspaces/CIS427_SocketProgrammingProject_Group2 (fix-logic) $ ./client localhost 6543
Connected to server.
```

```
@jahanCl ➜/workspaces/CIS427_SocketProgrammingProject
The server disconnected.
@jahanCl ➜/workspaces/CIS427_SocketProgrammingProject
Connected to server.

1. Login
2. Quit
Selection: 1
Username: Jahan
Password: password123
200 OK|userId=1

Discussion Board Menu:
1. View questions
2. View question and answers
3. Post question
4. Post answer
5. Logout
6. Quit
Selection: 1
COUNT|value=0
END
```

Status: Pass

---

8. Defects and Testing Limitations

No assertion failures were reported in the nine automated groups in either run.

Testing limitations include:

- Tests ran on localhost in one Codespace.
- The measured load used up to ten clients.
- The workload was short rather than a long-duration soak test.
- The Python script tests server behavior directly. Client interface and server shutdown behavior were evaluated through separate manual tests.

- Oversized and non-ASCII requests are not covered by this script and should be documented through additional tests.
- Passing these tests does not prove the absence of every race condition or resource leak.

9. Conclusion

All nine automated test groups passed. DiscussU enforced the tested session rules, rejected malformed requests, recovered after errors, reconstructed fragmented and batched messages, and allowed connected clients to share questions and answers. Concurrent answers were retained with unique IDs, and surviving clients continued working after another client disconnected unexpectedly.

In the main load test, all 200 posting requests from ten clients succeeded, and every created question was subsequently retrieved. The average completed-request response time was 2.355 milliseconds. These results support the application's behavior under the tested workload, while manual tests and the documented limitations provide the remaining context for the final assessment.

Scalability Analysis

1. Expected behavior with 10 clients.
    a. The application will run just as intended, allowing each to login, post questions and answers and view others questions and answers.
2. Expected behavior with 100 clients.
    a. The application will run as it should, however with so many clients the amount of questions and answers may become overwhelming.
3. Likely Bottlenecks
    a. The more users there are and the more questions and answers posted the more memory the system will use which could slow the application down.
4. Memory usage per client.
    a. Each client uses about 1.25 Gb of memory.
5. Thread or task growth.
    a. The server creates one thread per connected client. Therefore, 100 connected clients would require approximately 100 handler threads, in addition to the main server thread. Completed thread objects are retained until shutdown, so repeated connections also increase bookkeeping overhead. A thread pool could improve resource management at larger scales.
6. Logging overhead.
    a. Every request generates log activity. Log writes are protected by a mutex and flushed immediately, which makes entries available quickly but adds file writing overhead. As activity grows, logging could become a bottleneck. Buffered logging and log rotation would be useful improvements.
7. Shared data contention.
    a. This mostly occurs when multiple users try to post and answer to the same question at the same time. We handled this by making use of threads with our handle client.
8. Network bandwidth.
    a. Upload Speed is 1778.84 Mbits/s Download Speed is 567.00 Mbits/s
9. Persistent Storage Limitations.
    a. Our application is able to store information and retain it even after the client side has been discontinued as long as the server side is still running. However if the server side stops the information is lost.
10. Single Failure Server Risk.
    a. Our application does suffer from a single server. If the server were to go down at any point all of the questions and answers stored in it would be lost.
11. Changed needed to support hundreds or thousands of users
    a. Implementing a searching system would definitely be recommended as there would be so many questions and answers. We would definitely want to find a way

to save server side data since if the server went down that would be a lot of information lost.

Future Improvements

1. Database backed persistence: This would be a great idea to implement. This way if the server ever went down we would not lose all of the information stored there.
2. Searching System: Being able to search for questions or answers based on the author, body, or title would be very beneficial especially if the application was dealing with lots of clients.
3. Improved concurrency: Use a thread pool or asynchronous networking to manage more connections efficiently.

Ripple Report

Effects on Users

1. Which feature created the greatest value for users?
    - The creation of the menu and the numbered commands. This way the user only had to type in a number from a menu of options to say what they wanted to do, instead of typing in a whole line of code they'd have to remember off the top of their head.
2. Which design decision most improved reliability?
    - When we were able to make it so the information was saved in the server. This way we could reliably store information there without losing it when the client side shut down.
3. Which part of the interface was most difficult for users to understand?
    - The most complicated part would probably be viewing the answers of a question. It is more complicated than the others because you need to input specific information about the question you want to see the answers to.
4. How did error handling influence user trust?
    - The more errors that were fixed improved user trust because it was clear that each time the application improved substantially.
5. What user assumption turned out to be inaccurate?
    - We assumed that users would be tech savvy enough to use the dashboard, and we were right.
6. What evidence suggests the application solves the intended problem?
    - While we were using it we were able to respond to each other's questions and post our own. If this were in a class scenario we would have been able to ask questions we had and gotten them answered.

Effects on Teammates

1. How did the protocol design affect parallel development?
    - It helped us since we had more of a map that we both knew we were working towards. So even when we were working separately we were still both working towards the same thing.
2. Which documentation helped teammates work independently?
    - The protocol specification was really helpful in making the code since we were able to reference agreed upon commands and error codes we had already decided on. Thus The codes we wrote were able to work with each other with much less editing then there would have been.
3. Where did unclear responsibilities or assumptions create rework?

    - Rework was needed when it was unclear if we were making the code with the expectation that the user would be entering the raw commands or not. We had to rework the code when we realized it needed to be much more user friendly.
4. How did the team resolve conflicting technical ideas?
    - We considered what would make the best and most user friendly result and tried to work towards the option that pushed us closer to that.
5. What would the team change about its development process?
    - We would start working with the intent of user friendly formatting from the very start, as that required a lot of rework about halfway through the project.

Effects of Networking Decisions

1. Message framing: We decided to use '|' to frame the different parts of the commands. We did this because it is a pretty uncommon character used and it was easy to read to be able to separate the different parts. The benefit was that we were able to use that to have the code read only certain parts of the command in order to get the value of certain variables like author, title, and body. WE may use a different character in the future since if a user put '|' in their message it would confuse the code.
2. Synchronization: We made it so that the server would constantly be accepting input from clients. We did this so that multiple clients would be able to affect the same data at the same time, like posting answers to the same question. Doing this does increase the memory usage of the application. I don't think we would do it differently in the future though as it is necessary to allow multiple clients to access the same data at the same time.
3. Data persistence: We made it so that data is stored in the server side of the application. Before the information was not being saved if it stopped running and we needed to fix that. We did this so that when the client side of the application stops running the information is saved. Unfortunately, if the server side of the application stops running the information is still lost, so we have not achieved complete data persistence quite yet. In the future we would work so that if the server stopped running it would keep the data saved.

Scale Reflection

If this application were to be used by hundreds or thousands of users the first things I would improve is persistence, performance, and reliability. In order to improve persistence the main step that would need to be taken is to ensure that if the server ever shuts down the data saved there is retained. With so many people using the application a server shutdown right now would be devastating as it would mean that all the questions and answers all those people have entered would be lost. Performance would also be a main problem that would need to be

addressed quickly. With so many people using the application the amount of memory it would be using would be very high and thus would slow down the application a lot. We would take steps to reduce the memory usage of each client. Finally, reliability is an important consideration. With that many people posting questions and answers it is very possible that a lot of that information will not be entirely accurate. We would attempt to introduce a kind of fact checker into the system. Perhaps allowing a professor to mark false answers and mark some answers as correct and well explained. This would not only allow the information gathered from our application to be much more accurate but would also heavily increase user trust.

AI Usage

What AI tools has the team used for this project, if any?

The team used **U-M GPT** during this project

2. **How did the team use them? Give one or two concrete examples (for example, brainstorming, understanding socket APIs, designing the protocol, writing or debugging code, writing tests, or writing documentation).**

We used U-M GPT to help us understand the socket application programming interface (API) because neither team member had previously created a TCP client-server application. For example, we asked it to explain the purposes of functions such as socket(), bind(), listen(), accept(), connect(), send(), and recv().

We used it to help design and understand the client socket handler. Much of the initial socket-handler code was generated with AI assistance. Of course, we reviewed and modified this code to match our application protocol and project requirements.

3. **How much did the tools help the team's productivity?**

U-M GPT significantly improved our productivity. It helped us create initial code for sockets rather quickly. It helps us understand unifiliar code. Without this assistance, we would have spent more time researching unfamiliar socket functions and debugging basic networking behavior.

4. **How much did they help, or hinder, the team's learning of the socket programming and networking concepts in this project?**

The tool helped us learn by explaining unfamiliar code and networking concepts to us.
However, generated code was sometimes more complicated than our project required. Because we were new to these concepts, we spent considerable time asking follow-up questions to understand the complex and ask how to simplify it.

Individual Reflection

Kieran Fix
The specific task I completed were:

- Curiosity Report
- Problem statements and value propositions
- Final requirements
- Error handling strategy
- Known limitations
- Scalability analysis
- Multi client simultaneous access

The most important concept I learned was utilizing two sides of an application. Such as server and client side. A technical challenge I faced was understanding the threads that we used to allow multiple clients to access the same data at the same time.I want to improve my skills with memory usage. My work allowed the format of the code to be more readable and user friendly.

Jahan Clare
While we both contributed to writing code. I specifically contributed to the architecture sequence diagrams, the data-structure design, and the testing reports. Through this project, I learned what sockets are and how they allow clients and servers to communicate over TCP.
Testing was the most challenging part for me. I had to learn how to create meaningful test cases, compare expected and actual results, and gather evidence showing whether the application worked correctly. This helped me get better at testing at made me learn their so many things that can go wrong with code
In future projects, I want to improve my initial design process and establish clearer goals for what the finished application should look like. Because I felt like we were all other places deciding on what to actually implement and that made it hard to decide what to prioritize making. I also wanted to become more confident in technical report writing, which was a significant struggle for me throughout this project.
My work contributed to user value by checking that the application behaved as intended and documenting its results and limitations. Students need to be able to trust that their questions and answers are stored correctly and that errors do not disrupt the discussion board.